08 readonly (AND CONSTRUCTOR PARAMETER PROPERTIES)

`readonly` is a TypeScript modifier meaning: this property can only be assigned ONCE — inside the constructor — and never reassigned anywhere else in the class. It's a compile-time-only guarantee; it does not exist in the compiled JavaScript at all.

REAL EXAMPLE FROM THIS PROJECT

```
// task.controller.ts
export class TaskController {
  constructor(private readonly taskService: TaskService) {}
  ...
}
```

Writing `private readonly taskService: TaskService` as a CONSTRUCTOR PARAMETER does two things in one line — this is TypeScript shorthand called a "parameter property":

1. Declares a class field `taskService`.
2. Assigns it: `this.taskService = taskService`.

Written the long way, it's exactly this:

```
export class TaskController {
  private readonly taskService: TaskService;

  constructor(taskService: TaskService) {
    this.taskService = taskService;
  }
}
```

THE TWO MODIFIERS, SEPARATELY

private	— VISIBILITY. `this.taskService` can only be accessed from inside TaskController itself, not from external code holding a TaskController instance.
readonly	— MUTABILITY. Once set in the constructor, nothing else in the class can reassign it (`this.taskService = X` becomes a compile error anywhere outside the constructor).

They're independent — you can have `private` without `readonly`, `readonly` without `private` (public but unchangeable), or both, as here.

WITHOUT readonly vs WITH readonly — COMPILED AND CHECKED

```
// WITHOUT readonly — compiles FINE, no error at all
class TaskControllerNoReadonly {
```

```
  constructor(private taskService: TaskService) {}

  someMethod() {
    this.taskService = new TaskService();   // allowed!
  }
}

// WITH readonly — compile ERROR
class TaskControllerWithReadonly {
  constructor(private readonly taskService: TaskService) {}

  someMethod() {
    this.taskService = new TaskService();
    // TS2540: Cannot assign to 'taskService' because it is a
    // read-only property.
  }
}
```

(Verified with `tsc --noEmit --strict` on both versions — the second one is the only one that actually fails to compile.)

WHY THIS MATTERS FOR DEPENDENCY INJECTION

Nest's DI container (see theory/07-dependency-injection.txt) creates ONE instance of TaskService and hands it to TaskController's constructor. `readonly` guarantees that injected instance can never be silently swapped out later by a stray `this.taskService = new TaskService()` somewhere else in the class — which would replace Nest's managed singleton with an un-managed, disconnected instance (breaking the whole point of DI) without so much as a compiler warning if `readonly` weren't there.

`readonly` has ZERO effect on Nest's actual injection mechanism at runtime — Nest doesn't check for it. It's purely a TypeScript safety rail for whoever edits this class later.

★ RULE OF THUMB

Any constructor-injected dependency (`private readonly xService: XService`) should almost always be `readonly` — you never legitimately need to reassign an injected dependency after construction. Reach for plain `private` (no `readonly`) only for fields that are genuinely meant to change after the constructor runs (e.g. mutable internal state like `private tasks: Task[]` in task.service.ts, which gets pushed to/spliced throughout the class's lifetime).

See also: theory/07-dependency-injection.txt, theory/06-decorators.txt